

```

    }

    PieDataSet pieSet = new PieDataSet(pieEntries, label: "Expenses by Category");
    pieSet.setColors(ColorTemplate.MATERIAL_COLORS);
    pieSet.setSliceSpace(2f);
    pieSet.setValueTextSize(12f);
    pieDataLive.postValue((new PieData(pieSet)));

    List<BarEntry> barEntries = new ArrayList<>();
    List<String> labels = new ArrayList<>();
    if (data != null && data.get("budgets") instanceof List) {
        List<Budget> budgets = (List<Budget>) data.get("budgets");
        if (budgets != null && !budgets.isEmpty()) {
            for (int i = 0; i < budgets.size(); i++) {
                Budget b = budgets.get(i);
                float spent = (float) b.getSpentAmount();
                float remaining = (float) Math.max(0.0, b.getTotalAmount() - b.getSpentAmount());
                barEntries.add(new BarEntry(i, new float[]{spent, remaining}));
                labels.add(b.getTitle() == null ? ("Budget " + (i + 1)) : b.getTitle());
            }
        }
    }

    if (barEntries.isEmpty()) {
        barEntries.add(new BarEntry(0, new float[]{0f, 1f}));
        labels.add("No Budgets");
    }

    BarDataSet barSet = new BarDataSet(barEntries, label: "Budget Spent vs Remaining");
    barSet.setStackLabels(new String[]{"Spent", "Remaining"});
    barSet.setColors(ColorTemplate.MATERIAL_COLORS);
    pieSet.setValueTextSize(10f);
    BarData barData = new BarData(barSet);
    barData.setBarWidth(0.6f);
    barDataLive.postValue(barData);
    barLabelsLive.postValue(labels);

```

S- Single Responsibility

The DashboardViewModel has a single responsibility. It manages and prepares the incoming data for the dashboard. It does not deal with UI, navigation, or any user interaction. By separating this from Dashboard Fragment, we have made the code much easier to maintain and test. Changes that are made to the charts in terms of calculations or budget aggregation will only affect the ViewModel, not the fragment.

```

public void updateCharts(Map<String, Object> data) { 3 usages
    List<PieEntry> pieEntries = new ArrayList<>();
    if (data != null && data.get("categories") instanceof Map) {
        Map<String, Double> categories = (Map<String, Double>) data.get("categories");
        if (categories != null && !categories.isEmpty()) {
            for (Map.Entry<String, Double> e : categories.entrySet()) {
                double v = e.getValue() == null ? 0.0 : e.getValue();
                pieEntries.add(new PieEntry((float) v, e.getKey()));
            }
        }
    }
}

```

O- Open/Closed Principle

The ViewModel is open to extension, but closed for modification. If we wanted to add a different chart, we could use and extend the same updateCharts logic/method for a new chart. This can all be done without modifying the code that already deals with the pie chart or bar chart data. This ensures that any addition of new charts will not affect the functionality of the existing code.

```

dashboardViewModel =
    new ViewModelProvider(requireActivity()).get(DashboardViewModel.class);

dashboardViewModel.getDashboardData().observe(getViewLifecycleOwner(), data -> {
    dashboardViewModel.updateCharts(data);
});

```

D- Dependency Inversion Principle

The fragment depends on DashboardViewModel instead of chart logic or data sources. The fragment observes LiveData and is not affected by how the data is obtained or processed. In doing this, we are able to either replace DashboardModel or change how the Firebase data logic works without having it affect the fragment, or having us change the fragment logic. This allows the code to be flexible and testable.